portcast

Sr. Software Engineer Take-Home: Distributed Rate Limiting

Guidance

This assignment is designed to take about 4 hours. Please submit within 3-5 days of receiving it. If you need more time, reply to the original email with a new deadline.

Use Python 3. AI assistance is allowed. Please indicate which parts were AI-assisted and which were your own design decisions. This will be considered in evaluation.

The Problem

A service team at our company is about to launch a public API and needs to enforce per-client rate limits across multiple instances of their service. They've asked you to design and build the rate limiting component.

The limits must be enforced consistently across all instances of the service, not per-instance. Different clients and different routes need different limits. The team expects the API to handle thousands of requests per second across the fleet.

The Workload

The service team has shared the following expectations about their API at launch and over the next 12 months.

Clients. Around 5,000 active API clients at launch, expected to grow to 50,000 within a year. Clients fall into three tiers (free, standard, enterprise) with different limits. A small number of enterprise clients drive a disproportionate share of traffic.

Traffic. Average sustained load of around 3,000 requests per second across the fleet, with peaks of 15,000 requests per second during business hours in major timezones. Traffic is bursty at the per-client level: a single client may send 100 requests in a second and then nothing for a minute.

Routes. Roughly 40 distinct API routes, with limits varying by route. The 5 most popular routes account for around 80% of traffic.

Deployment. The service runs as a horizontally scaled fleet, currently 8 instances, expected to grow. Instances come and go during deploys and autoscaling events.

Latency budget. The service team's own p99 latency target is 200ms. Your rate limiter sits in the request path. They've told you that anything above 10ms of overhead per request will be a problem.

portcast

You don't have to design for the 12-month projection on day one, but your DESIGN.md should explain what would need to change to get there.

What You Decide

You decide how the rate limiting component is integrated: a sidecar, a shared service, a library, middleware, something else. You decide how limits are configured and how the configuration is loaded. You decide what the integration surface looks like for the service team consuming your work. You decide the storage, the algorithm, the failure semantics, and the deployment shape.

We expect a working system that another engineer could pick up and use, demonstrated end-to-end with at least one consumer of your rate limiter running under load.

Deliverables

A repository containing your implementation, a way to run it (we should be able to bring up the whole demo with a single command), a way to load test it that produces real numbers, and tests.

A DESIGN.md that explains what you built and why. We want to see the decisions, not a feature list. At minimum we want to understand: how you chose the integration shape, what algorithm you picked and what you rejected, how state is shared across instances, what happens when things go wrong, what your numbers actually are when you run the load test, and where your system falls over.

Evaluation

We will evaluate the quality of your decisions and how clearly you defended them, whether your code matches what your DESIGN.md claims, whether your tests exercise the actual behavior of your implementation rather than just the spec, whether your numbers are real and reproducible, the cleanliness of your code with no dead paths or unused surfaces, and your honesty about the limits of what you built.

Submission

Send a link to a git repository or a zip to g@portcast.io.

Disclaimer

This exercise is for technical skills assessment only. It is not related to Portcast's product and your work will not be used by the company.